Reliability-Aware Fault Injection for Microscaling-Based DNNs

Undergraduate Project (UGP) Final Report
CS496

Peeyush Agarwal
Roll No.: 230749
Department of Aerospace Engineering
Indian Institute of Technology Kanpur

Supervisor: **Prof. Urbi Chatterjee**
Indian Institute of Technology Kanpur

November 2026



Abstract

The adoption of microscaling formats in deep neural network (DNN) accelerators reduces the cost of low-precision inference, but their impact on DNN reliability under hardware faults remains largely unexplored. Unlike conventional floating-point representations, microscaling formats represent multiple values using a shared scale, making the effect of a bit fault dependent not only on the corrupted value but also on the scale and other values within the scaling block. We propose a reliability analysis methodology for microscaling-based DNNs that characterizes how faults in element values and shared scaling factors propagate through the network. The methodology evaluates fault sensitivity across different microscaling formats, block sizes, tensor types, and fault locations, and identifies the representation characteristics that make a model more vulnerable to faults. We conduct an extensive fault-injection study on representative CNN and Transformer models using weight and activation tensors. Our results show that reliability varies substantially across microscaling formats even when they provide comparable inference accuracy, and that faults affecting shared scaling information can have a significantly different impact from faults in individual elements.

The methodology has two parts: `mxfi`, a bit-accurate fault injector that flips real bits of element codes and shared scales, and a flip-case algorithm that follows a fault from the stored bit to the prediction and sorts every possible single-bit flip into one of 22 cases. It was applied to five element formats, four block sizes, weights and activations, and every bit and layer, on ResNet8, RepVGG-A0 and a vision Transformer, on CIFAR-10 and CIFAR-100. Two exhaustive campaigns (1.1 million injections) give exact ground truth. Formats with the same accuracy differ in vulnerability by up to three orders of magnitude, with `e3m2` < `e4m3` < `e5m2` on every network. A fault in a shared scale is always worse than a fault in an element, typically by 13× to more than 1000×. The flip-case algorithm explains why: three cases, flips into NaN/Inf codes and scale overflows, are under 2% of stored bits but cause more than half of all failures, and every one of them is a $0 \to 1$ flip. Read-time guards that follow from this, and cost no memory, cut element damage by 29–4100× on the larger models. Finally, using the case of each bit as a sampling plan recovers the exhaustive failure rate with about 1,000 injections instead of half a million.

Contents

1 Introduction **3**
1.1 Objectives . . . 3
1.2 Contributions . . . 3

2 Background **4**
2.1 Microscaling formats . . . 4
2.2 Two fault sites . . . 4
2.3 Related work and the gap . . . 5

3 Proposed Methodology **5**
3.1 Fault model and metric . . . 5
3.2 The `mxfi` fault injector . . . 5
3.3 The flip-case algorithm: how a fault propagates . . . 6
3.4 Using the cases to plan a campaign . . . 8

4 Experimental Setup **9**

5 Results and Analysis **10**
5.1 Validating the injector . . . 10
5.2 O2: Equal accuracy does not mean equal reliability . . . 10
5.3 O1: Shared scales dominate . . . 11
5.4 O1: How faults propagate, case by case . . . 11
5.5 O3: Block size is a trade-off . . . 12
5.6 O4: Tensor type and fault location . . . 13
5.7 O5: What makes a model vulnerable . . . 14
5.8 A harder task: CIFAR-100 . . . 15
5.9 New experiment E29: case-guided campaigns . . . 15

6 Methodological Findings **16**

7 Design Guidance **17**

8 Conclusion **17**

9 Limitations and Future Work **17**

Acknowledgements **18**

References **18**

1. Introduction

DNN accelerators are moving to narrow number formats, because memory traffic and arithmetic both get cheaper as the bit width falls. A 4- to 8-bit number cannot cover the range of a whole tensor on its own. *Microscaling* (MX) formats solve this by grouping values into blocks that share one scale factor [1, 2]. The OCP MX specification defines 8-, 6- and 4-bit variants (MXFP8, MXFP6, MXFP4), and they are now supported in commercial hardware.

Hardware is not fault-free. Radiation strikes, voltage droop and ageing can flip a stored bit, and in a DNN such a flip can silently change a prediction (silent data corruption, SDC). DNN reliability under bit flips has been studied for FP32 and integer formats [3, 4, 5]. Those studies assume that each value is its own word, so one flip changes one number. MX breaks that assumption: a flip in an element changes one value, but a flip in the shared scale changes every value in its block. How much this matters, and which properties of a format make a network fragile, had not been measured.

In this report, *reliability* means how well a network keeps giving the right answer when the hardware corrupts a stored bit. It is measured as the probability that one fault changes one prediction (Section 3.1). It is different from accuracy, which is measured without faults, and from robustness to noisy or adversarial inputs: here the inputs are clean and the fault is in the stored numbers.

1.1. Objectives

O1 Fault propagation: characterise how faults in element values and in shared scales propagate through the network, and quantify the difference.

O2 Formats: measure fault sensitivity across MXFP8, MXFP6 and MXFP4, and test whether reliability differs between formats of comparable accuracy.

O3 Block size: find how the scale-sharing granularity trades storage against reliability.

O4 Tensor type and fault location: compare weights with activations, scale bits with element bits, high-order with low-order bits, and layer with layer.

O5 Vulnerability drivers: identify the representation characteristics that make a model more vulnerable, and what would protect it.

1.2. Contributions

- `mxfi`, an open-source, bit-accurate fault injector for MX formats, validated against two exhaustive campaigns (Section 3.2).
- A flip-case algorithm that traces a single-bit flip through five stages and sorts it into one of 22 cases. It is complete by exhaustive check, and it predicts which flips are catastrophic before any injection (Section 3.3).
- A characterisation along all four axes (format, block size, tensor type, fault location) on a CNN, a re-parameterised CNN and a Transformer, on two datasets (Section 5).
- Two read-time guards that need no extra memory, and a case-guided campaign plan that needs about a thousand injections per network (Sections 5.7 and 5.9).

2. Background

2.1. Microscaling formats

An MX block holds K values ($K = 32$ in the specification). It is stored as one 8-bit shared exponent X, the *scale*, and K narrow element codes m_i:

$$v_i = \text{decode}(m_i) \cdot 2^{X-127}, \qquad i = 1, \ldots, K. \tag{1}$$

The scale uses the E8M0 encoding: an unsigned power of two, with code 255 reserved for NaN. Each element is a small sign–exponent–mantissa number (Table 1). The OCP rule sets the unbiased shared exponent to $X - 127 = \lfloor \log_2 \max_i |v_i| \rfloor - e_{\max}$, where $e_{\max}$ is the exponent of the element format's largest normal value (8 for `e4m3`), so the largest value of every block lands in the top binade of the element format.

Table 1: MX element formats. Only the 8-bit formats have special codes (NaN or $\pm\infty$).

MX format	Element	Bits (S/E/M)	Largest value	Special codes
MXFP8	`e4m3`	1/4/3	448	2 (NaN)
MXFP8	`e5m2`	1/5/2	57344	8 (2 Inf, 6 NaN)
MXFP6	`e3m2`	1/3/2	28	none
MXFP6	`e2m3`	1/2/3	7.5	none
MXFP4	`e2m1`	1/2/1	6	none

2.2. Two fault sites

A single-bit flip can land in one of two places (Figure 1):

- **Element fault.** One code m_i changes, so one value changes. In `e4m3` and `e5m2` the new code can be NaN or infinity.
- **Scale fault.** Flipping bit j of X moves the exponent by exactly $\Delta = \pm 2^j$, so all K values are multiplied by 2^{Δ}. Bit 7 gives $\Delta = \pm 128$, and a flip that produces $X = 255$ turns the whole block into NaN.

A block takes $8 + Kw$ bits, of which only 8 are the scale (about 3% for $K = 32$, $w = 8$). But if every bit is equally likely to flip, the expected number of corrupted *values* is $8K$ from scale faults and Kw from element faults, a ratio of $8 : w$. An 8-bit format gets about half of its corrupted values through the 3% of storage that holds scales.

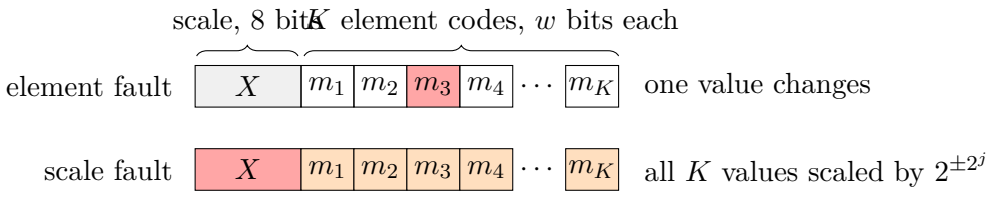


Figure 1: One MX block and its two fault sites. Red marks the flipped word, orange the values it changes.

2.3. Related work and the gap

Fault-injection tools such as Ares [4] and PyTorchFI [5] inject random flips and report a failure rate, and Li et al. [3] showed that errors propagate differently by layer, data type and bit position. Statistical fault injection [6] and value-aware sampling such as TreeFI [7] reduce the number of injections needed. Bit-flip attacks [8, 9] search for the most damaging bits. All of these target FP32 or integer formats. None models a shared scale, and none gives an exact map from a flip to its effect in MX, where the scale, the element, special codes and the datapath together decide whether a flip is harmless or catastrophic.

3. Proposed Methodology

The methodology has two parts. The fault injector (Sections 3.1–3.2) measures how often faults cause failures. The flip-case algorithm (Section 3.3) explains which faults cause them and why, and turns that into protection and a cheaper campaign plan.

3.1. Fault model and metric

I use the standard single-bit-flip model on stored values: one bit of one element code, or of one scale byte, is inverted. Weight faults are *persistent* (every inference reads the corrupted weight); activation faults are *transient* (one inference). Unless stated otherwise, every stored bit is equally likely to flip.

Each campaign runs a fixed, class-balanced set of 200 test images. For a fault f, the damage $d(f)$ is the fraction of predictions that change from the fault-free ("golden") MX model, and the failure rate is its mean over faults:

$$d(f) = \frac{1}{200} \sum_{n=1}^{200} \mathbb{1}\left[\hat{y}_n^{(f)} \neq \hat{y}_n^{\text{golden}}\right], \qquad \text{rate} = \mathbb{E}_f\left[d(f)\right]. \tag{2}$$

This *per-inference* rate is the probability that one fault corrupts one inference, and it is the only rate on which weights and activations can be compared. A NaN or ∞ output counts as changed. The common "did any image change" SDC rate is not used, because Section 6 shows it depends mostly on the chosen images.

3.2. The `mxfi` fault injector

Three design choices keep the injection faithful to what an accelerator executes:

1. **Deployment form.** Batch normalisation is folded into the convolution and RepVGG branches are fused before quantisation, so nothing downstream can re-normalise the damage.
2. **Code-domain injection.** Tensors are encoded into real MX bit patterns, the fault is applied to the stored code, and the block is decoded afterwards. This is what lets a scale fault change all K values and an element fault land on a NaN code.
3. **Faithful special values.** As in the OCP conversion, a NaN anywhere in a block makes the whole block NaN when activations are re-quantised. (An earlier version of the codec mapped NaN to 0, which hid activation damage; it was fixed and all activation results re-measured.)

Algorithm 1 Weight fault-injection campaign

Require: network M, format F, block size K, images D, budget n
1: $M \leftarrow \textsc{Deploy}(M)$ ▷ fold batch norm, fuse branches
2: **for all** quantisable layers ℓ **do**
3: split each row of W_ℓ into blocks of K; $X_\ell \leftarrow$ OCP scales; $C_\ell \leftarrow$ element codes
4: $W_\ell \leftarrow \textsc{Decode}(X_\ell, C_\ell)$ ▷ the network now holds MX values
5: **end for**
6: $\hat{y} \leftarrow \arg\max M(D)$ ▷ golden predictions
7: **for** $t = 1$ to n **do**
8: draw fault $f = (\ell, \text{site}, \text{index}, b)$ over all stored bits
9: flip bit b of C_ℓ[index] or X_ℓ[index]; re-decode 1 or K weights
10: $\hat{y}^{(f)} \leftarrow \arg\max M(D)$; record $d(f)$ and non-finiteness; restore
11: **end for**
12: **return** mean damage with confidence interval

Replacing the random draw in line 8 with an enumeration of every bit makes the campaign *exhaustive*. MX fault spaces are small enough for this on ResNet8: 638,624 injections for `e4m3` and 483,904 for `e3m2`. These two campaigns are the ground truth used to validate the sampled campaigns and the flip-case algorithm. Activation faults are injected through forward hooks that quantise a layer's input, flip one bit of one image's encoding, and decode it.

3.3. The flip-case algorithm: how a fault propagates

A campaign gives one number per configuration. To explain it, I trace a single flip through the chain in Figure 2. A flip becomes a wrong prediction only if all five stages pass it on; a flip that stops at any stage causes no failure.

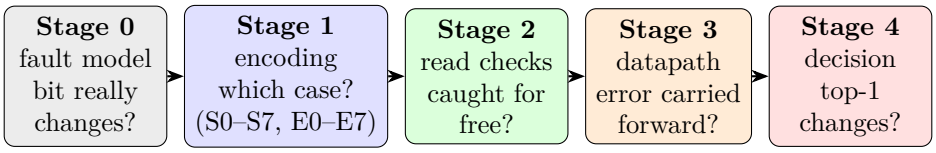


Figure 2: The five-stage chain from a stored bit to a wrong prediction. A final stage adds the damage over all faults, weighted by the probability of each flip direction.

Stage 0: fault model. A transient flip can go $0 \to 1$ or $1 \to 0$; a stuck-at fault matters only when the stored bit differs from the stuck value. Memories are often asymmetric (DRAM charge loss is mostly $1 \to 0$), so each flip is weighted by the probability of its direction.

Stage 1: the case tree. This is the core of the algorithm (Algorithm 2). For a scale flip, $X' = X \oplus 2^j$ and $\Delta = X' - X$; for an element flip, $c' = c \oplus 2^b$. The first matching condition gives the case. Table 2 lists all 22 cases.

Algorithm 2 Stage 1: the case of one single-bit flip

if the flip hits scale byte X, bit j **then**
  $X' \leftarrow X \oplus 2^j$; $\Delta \leftarrow X' - X$ ▷ block scaled by 2^Δ
  if $X' = 255$ **then return S1** to NaN
  else if $X = 255$ **then return S7** from NaN
  else if every element is ± 0 **then return S0** zero block
  else if $\Delta = +128$ or block overflows FP32 **then return S2** overflow
  else if $\Delta \geq 8$ **then return S3** grow $\geq 256\times$
  else if $\Delta > 0$ **then return S4** grow 2–16×
  else if $\Delta \geq -2$ **then return S5** shrink 2–4×
  else return S6 erase
  end if
else ▷ element code c, bit b
  $c' \leftarrow c \oplus 2^b$
  if c is NaN/Inf **then return E7** from special
  else if c' is NaN/Inf **then return E1** to special
  else if b is the sign bit **then return E0** if $v = \pm 0$, else **E2** negate
  else if exponent bit set **then return E3** grow (**E3a** from zero, **E3b** subnormal→normal)
  else if exponent bit cleared **then return E4** shrink (**E4a** to zero, **E4b** normal→subnormal)
  else if mantissa bit set **then return E5** grow (**E5a** from zero)
  else return E6 shrink (**E6a** to zero)
  end if
end if

Table 2: The 22 cases. Cases marked † (S1, S2, E1) are *catastrophic*: they create NaN or $\pm\infty$.

Scale byte X			Element code		
Case	**Condition**	**Effect**	**Case**	**Bit**	**Effect**
S0	all elements ± 0	masked	E0	sign of ± 0	masked
S1†	$X' = 255$	K NaNs	**E1**†	into NaN/Inf code	NaN or $\pm\infty$
S2†	$\Delta = +128$ or overflow	$\pm\infty$ or huge	E2	sign	$v \to -v$
S3	$\Delta \geq 8$	$\times 256$ or more	E3, E3a, E3b	exponent $0 \to 1$	grows
S4	$\Delta \in \{1, 2, 4\}$	$\times 2$ to $\times 16$	E4, E4a, E4b	exponent $1 \to 0$	shrinks
S5	$\Delta \in \{-1, -2\}$	$\div 2$, $\div 4$	E5, E5a	mantissa $0 \to 1$	small growth
S6	$\Delta \leq -4$	$\div 16$ or more	E6, E6a	mantissa $1 \to 0$	small shrink
S7	$X = 255$	prior fault only	E7	code already special	prior fault only

Worked examples. On the ResNet8 used for the exhaustive `e4m3` campaign the weight scales lie between 114 and 120. Take $X = 118 =$ `0111 0110`:

- bit 7 is 0; flipping it gives $X' = 246$, $\Delta = +128$, so the case is **S2** (every value in the block is multiplied by 2^{128});
- bit 3 is 0; flipping it gives $X' = 126$, $\Delta = +8$, so **S3** (block $\times 256$);
- bit 4 is 1; flipping it gives $X' = 102$, $\Delta = -16$, so **S6** (block divided by 2^{16}, almost erased).

For an `e4m3` element (sign, 4 exponent bits, 3 mantissa bits, bias 7), the largest value 448 is `0 1111 110`. Flipping its last bit gives `0 1111 111`, the NaN code, so the case is **E1**. The

value 1.0 is `0 0111 000`; flipping the top exponent bit gives `0 1111 000` $= 2^{15-7} = 256$, case **E3**; flipping the sign bit gives -1.0, case **E2**. The same flip position can therefore fall in different cases depending on the stored code, which is why the case is computed from (code, bit) and not from the bit position alone.

Theorem 1 (Completeness)**.** *For each of the five element formats, every (code, bit) pair of an element and every (byte $X \in \{0, \ldots, 255\}$, bit j) pair of a scale, in zero and non-zero blocks, falls into exactly one case.*

Argument and check. Each branch tests its conditions in a fixed order and ends with an unconditional case, so every flip gets exactly one case. The input space is finite, so this was also checked by brute force over all codes, bits and scale bytes of all five formats (38 unit tests, all passing). □

Proposition 1 (Catastrophic flips are $0 \to 1$)**.** *Every flip in case E1, S1 or S2 sets a bit that was* 0*.*

Proof. NaN and Inf codes are runs of 1s: `S.1111.111` in `e4m3`, exponent `11111` in `e5m2`, and $255 =$ `11111111` for E8M0. A finite code has at least one 0 in that run, and one flip reaches the all-ones pattern only by setting that 0. S2 needs the block to grow, $\Delta = X' - X > 0$, and $\Delta = +2^j$ exactly when bit j was 0. □

Stage 2: free read-time checks. A clean OCP tensor obeys rules that can be checked when it is read, with no extra memory. **D1**: no element code is NaN/Inf; guard **G1** decodes such a code as 0, so an E1 fault only zeroes one weight. **D2**: every scale lies inside its tensor's clean band $[X_{\min}, X_{\max}]$, known offline for weights; guard **G2** zeroes a block whose scale is above the band, so S1/S2/S3 become a mild erase. **D3**: every non-zero block has an element in the top binade.

Stages 3–4: datapath and decision. With IEEE arithmetic a NaN survives every operation, even $\text{NaN} \cdot 0$. A ReLU written as `x > 0 ? x : 0` turns NaN into 0; $+\infty$ passes any ReLU. A finite error reaches the next layer as about $\delta \cdot a$ for an element (weight error times input activation) and $(2^{\Delta} - 1)P_B$ for a scale, where P_B is the block's partial sum. The outcome is no change, a silent wrong answer, or a non-finite output.

3.4. Using the cases to plan a campaign

Stage 1 needs no injection: the case of every stored bit can be counted from the quantised weights. This makes the algorithm a sampling plan. Let stratum h hold the faults with one case and one direction, with exact size N_h and weight $W_h = N_h/N$. Injecting n_h faults per stratum gives

$$\widehat{\text{rate}} = \sum_h W_h \, \bar{d}_h, \qquad \widehat{\text{Var}} = \sum_h W_h^2 \, \frac{s_h^2}{n_h} \left(1 - \frac{n_h}{N_h}\right). \tag{3}$$

Masked cases (E0, S0) have damage 0 and are skipped, and the rare catastrophic strata are no longer left to chance.

4. Experimental Setup

Table 3: Networks, all trained from scratch. Accuracies are FP32 top-1. The ViT has the embedding width and head count of DeiT-Tiny, depth 6 and 4×4 patches.

Network	Type	Params	CIFAR-10	CIFAR-100
ResNet8 [10, 11]	CNN	78 K	87.0%	58% (3 seeds)
ResNet8 width sweep	CNN	20–692 K	10 nets per width	–
RepVGG-A0 [12], fused	re-param. CNN	7.0 M	91.4–91.5% (3 seeds)	70–71% (3 seeds)
ViT [13, 14]	Transformer	2.7 M	80.4–81.9% (3 seeds)	47–49% (3 seeds)

- **Representation axes:** formats `e4m3`, `e5m2`, `e3m2`, `e2m3`, `e2m1`; block sizes $K \in \{8, 16, 32, 64\}$ (default 32); weights and activations; every scale and element bit; every layer.
- **Campaigns:** 200 test images per campaign; 3,000 sampled injections unless stated. In total about 1.44 million sampled faults in 119 weight campaigns on CIFAR-10, the two exhaustive campaigns (1.12 million), 390,000 multi-bit injections, and further activation and CIFAR-100 campaigns.
- **Datasets:** CIFAR-10 and CIFAR-100. ImageNet was not reachable from any machine available, so CIFAR-100 (ten times the classes, smaller margins) stands in for a harder task.
- **Compute:** a 4-core laptop CPU, and an NVIDIA A100 server at IIT Kanpur for the width sweep, CIFAR-100 and the exhaustive campaigns.

Table 4: Experiments and the objectives they answer.

Experiments	What they measure	Objective
E00–E02, E06	accuracy, format campaigns, the two exhaustive campaigns	O2, validation
E03	block size, under the OCP and a non-clipping scale rule	O3
E04	activation faults on all networks	O4
E08–E14, E16, E23	format geometry, sensitivity and margins, read-side NaN guard	O5
E24	every headline result repeated on CIFAR-100	all
E12, E15, E17–E22	metric, severity measure, sampling, multi-bit faults	method
E25–E28	flip-case algorithm against the exhaustive campaigns	O1, O5
E29	case-guided campaigns and guards on three networks	O1, O5

5. Results and Analysis

5.1. Validating the injector

Under `e4m3`, the exhaustive ResNet8 campaign gives a true any-image SDC rate of 0.4171; an ordinary 3,000-injection campaign returned 0.4270 [0.4094, 0.4448], containing the truth. Under `e3m2` the exhaustive rate is 0.1079 and the sampled estimate 0.1070 [0.0964, 0.1186]. The two exhaustive campaigns were recorded on two different trained copies of ResNet8, so they are used as two references, never as a comparison between formats.

5.2. O2: Equal accuracy does not mean equal reliability

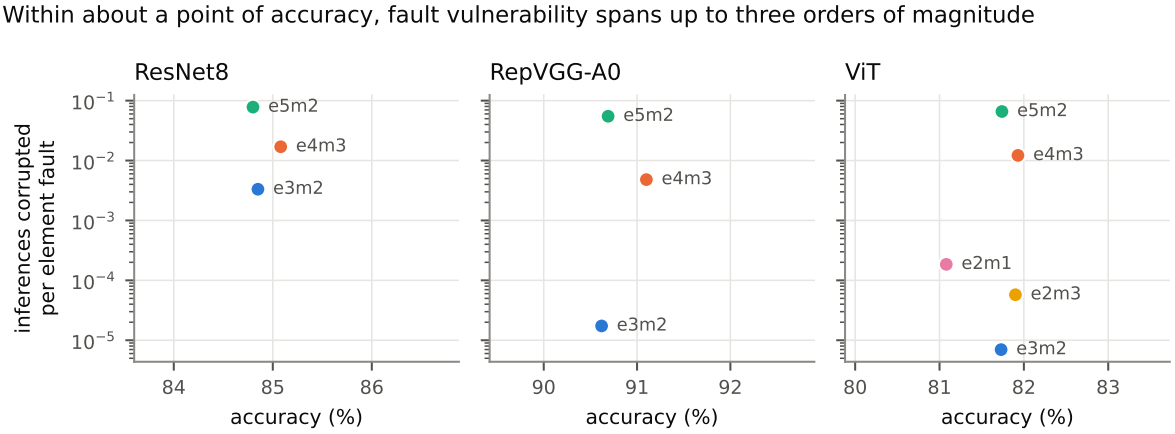


Figure 3: Accuracy against element-fault vulnerability (CIFAR-10, $K = 32$). Formats within about one point of accuracy differ by up to three orders of magnitude.

On the Transformer, `e5m2` and `e3m2` differ in accuracy by 0.01 points (81.74% and 81.73%), yet one element fault in `e5m2` is about 2,000 times as likely to corrupt an inference (Figure 3, Table 5). The order `e3m2` < `e4m3` < `e5m2` holds on all eight CIFAR-10 configurations with disjoint 95% intervals, and on all nine CIFAR-100 networks. Accuracy and bit width are not a guide to reliability.

Table 5: Per-inference rate of element faults ($K = 32$, CIFAR-10, mean over the seeds with a campaign). Lower is better. Five of the eight configurations are shown; widths 16, 24 and 32 follow the same order.

Network	**Seeds**	`e3m2` (6-bit)	`e4m3` (8-bit)	`e5m2` (8-bit)
RepVGG-A0	3	0.00001	0.00688	0.05381
ViT	3	0.00003	0.01258	0.06897
ResNet8	1	0.00332	0.01699	0.07801
ResNet8, width 8	5	0.01003	0.02005	0.08143
ResNet8, width 48	4	0.00035	0.01302	0.07812

5.3. O1: Shared scales dominate

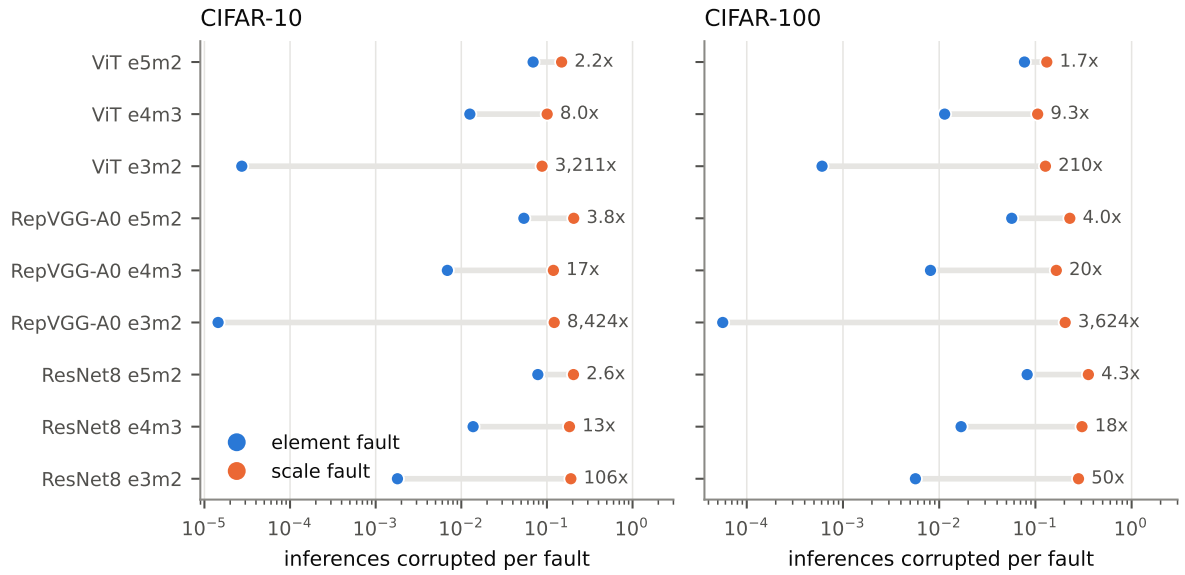


Figure 4: Per-inference damage of a scale fault (orange) and an element fault (blue). The scale is worse on every network, format and dataset.

A scale fault is worse than an element fault on every architecture, format and dataset (Figure 4). In the exhaustive ResNet8 `e4m3` campaign, 94.4% of scale flips change at least one prediction, and a scale fault corrupts on average 49.9 of 200 inferences against 3.3 for an element fault (15×). On the larger models the ratio reaches 10^3–10^4 for `e3m2`; these are lower bounds, because the `e3m2` element rate there is close to zero. It is smallest for `e5m2` (1.7–4.3×), whose element faults already destroy inferences through NaN codes. Scale faults also give non-finite outputs in 7–13% of cases *for every format*, including `e3m2`, because the scale has its own NaN code and its top bit is worth 128 exponent steps. The scale needs protection even when the element format has no special codes.

5.4. O1: How faults propagate, case by case

Labelling all 1.1 million exhaustive faults with their case (E25) shows where the damage comes from (Figure 5). In `e3m2`, S2 scale overflows are 0.5% of faults and cause 57% of all damage, with mean damage 0.97: almost every image fails. In `e4m3`, E1 flips into the NaN code are 1.2% of faults and cause 52% of damage, each with damage exactly 1.0. The ordinary element cases (E2–E6 and their sub-cases) are about 96% of faults but only 12–15% of damage.

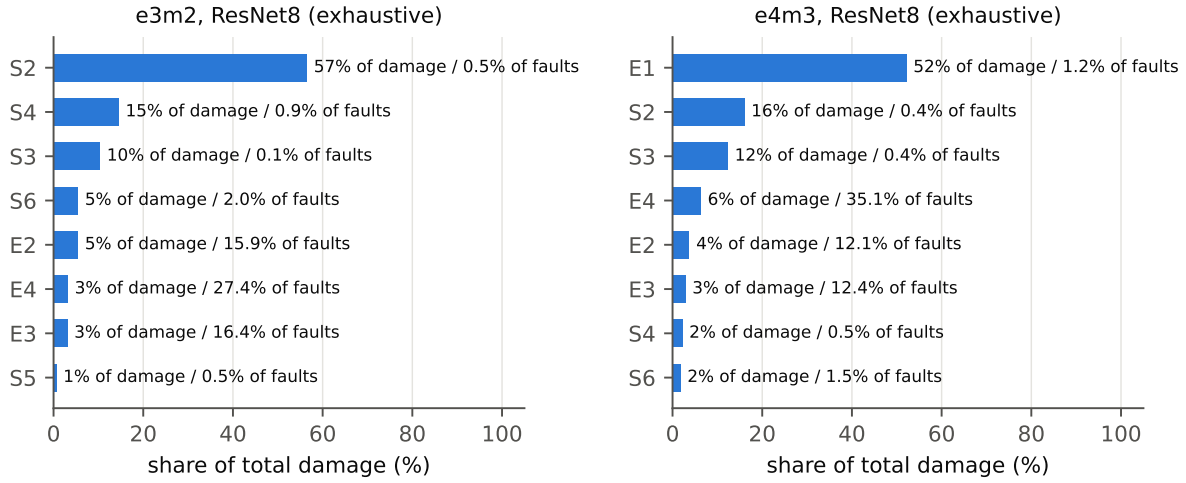


Figure 5: Share of total damage per case on the two exhaustive ResNet8 campaigns (top 8 cases), labelled with the share of damage and the share of faults.

The case explains 91–97% of the variance in damage (two-fold held-out R^2, against 9–22% for the fault site alone). It does not explain small finite errors: within ordinary element flips it explains 2–5%, and what matters there is the size of the error reaching the next layer, $|v' - v|\, 2^{X-127}$ times the input activity, which raises it to 64–67%.

Direction. As the proposition predicts, every catastrophic fault is a $0 \to 1$ flip (2,468 of 2,468 in `e3m2`; 10,321 of 10,321 in `e4m3`), and $0 \to 1$ flips cause 87.5–88.7% of all damage. The rate from $0 \to 1$ flips alone is 7.0–7.8× the rate from $1 \to 0$ flips. A memory whose upsets are mostly $1 \to 0$ would therefore see about 7–8× fewer failures than the usual symmetric model assumes.

5.5. O3: Block size is a trade-off

Block size barely changes accuracy (about one point from $K = 8$ to 64 on ResNet8), so it is a clean reliability axis, and the two fault sites move in opposite directions along it (Table 6). Larger blocks cut the scale's share of storage from 11.1% to 1.7%, so a scale is less likely to be hit, but each scale fault reaches more values and gets worse. Element faults become milder. Small blocks localise scale damage at the cost of storage; large blocks are safe if the few scale bytes are protected.

Table 6: Block size on ResNet8 (`e4m3`).

	$K = 8$	$K = 16$	$K = 32$	$K = 64$
Scale bits, share of storage	11.1%	5.9%	3.1%	1.7%
Scale fault, inferences corrupted (of 200)	40.1	45.3	48.7	50.1
Element fault, per-inference rate	0.0241	0.0208	0.0159	0.0136

The OCP rule can clip the block maximum, and smaller blocks have more maxima to clip, so it distorts block-size trends (element-side any-image SDC varied 2.2–7.3× under the OCP rule against 1.04–1.5× under a non-clipping rule). The trend above holds under both.

5.6. O4: Tensor type and fault location

Weights and activations. Per inference, an element fault in a weight does 0.6–2.2× the damage of one in an activation (Table 7); under `e4m3` the two are statistically indistinguishable. In both, about 1% of faults land on a special code and destroy the inference, and scale faults dominate (14× element in activations, 11× in weights, on ResNet8). The real difference is exposure: a weight fault persists across every inference until the memory is rewritten; an activation fault lives for one.

Table 7: Per-inference rate of element faults in weights and activations (CIFAR-10, $K = 32$, one network per row, so the weight column can differ from the seed means of Table 5).

Network	Format	Weight	Activation [95% CI]	Ratio
ResNet8	`e4m3`	0.0170	0.0122 [0.0077, 0.0182]	1.4×
RepVGG-A0	`e4m3`	0.0048	0.0075 [0.0049, 0.0101]	0.64×
ViT	`e4m3`	0.0122	0.0114 [0.0080, 0.0150]	1.07×
ResNet8	`e5m2`	0.0780	0.0359 [0.0302, 0.0421]	2.2×
ViT	`e5m2`	0.0659	0.0486 [0.0420, 0.0554]	1.36×

Bit position. Figure 6 gives the exhaustive damage per bit. Among faults that leave the output finite, the sign bit is the worst element bit. In total damage it ranks seventh of eight in `e4m3`, because most damage comes from flips into the NaN code, and a sign flip can never make one. In the scale, what matters is which bits are normally set: the scale bytes lie between 114 and 120, so bits 4–6 are always 1 and bits 3 and 7 almost always 0. Flipping bit 3 multiplies the block by 256 (75% of inferences corrupted) and bit 7 overflows it (98%); flipping bits 4–6 shrinks it (3%). Growing a block is catastrophic and shrinking it is mild, which is exactly the S2–S4 versus S5–S6 split of the case tree.

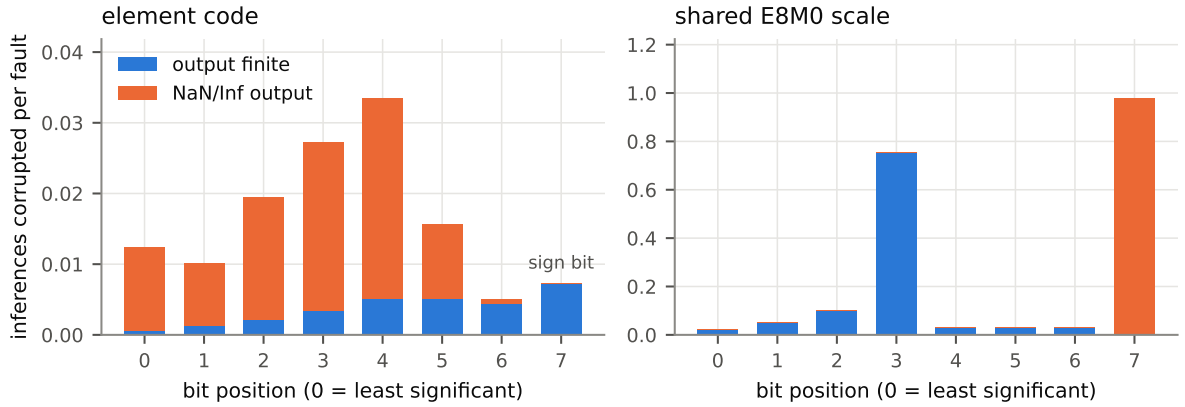


Figure 6: Exhaustive per-bit damage (ResNet8, `e4m3`, 638,624 injections), split by whether the output stayed finite.

Layer. Vulnerability runs opposite to layer size. The 3,584-bit stem convolution is the most fragile layer (any-image SDC 0.853) and the 304,128-bit final convolution the most robust (0.289). A uniform campaign spends 48% of its budget on the most robust layer and 0.6% on the most fragile; protection should follow vulnerability, not size.

5.7. O5: What makes a model vulnerable

Special codes. Only formats with NaN/Inf codes can turn an element flip into a non-finite value, and nothing downstream absorbs a NaN. On the Transformer 94% of `e5m2` element failures were non-finite, on RepVGG 92%; no `e3m2` element fault produced one in any campaign. In `e4m3` only 1.27% of element faults reach the NaN code, but they carry 78% of element damage on ResNet8.

Capacity absorbs perturbations, not NaNs. The width sweep isolates this (Figure 7). Across a 35× range of parameters the non-finite rate stays flat, but the format with no special codes (`e3m2`) improves 25×, `e4m3` (2 codes) 1.5×, and `e5m2` (8 codes) not at all. The benefit of a bigger model falls in exact order of the number of special codes.

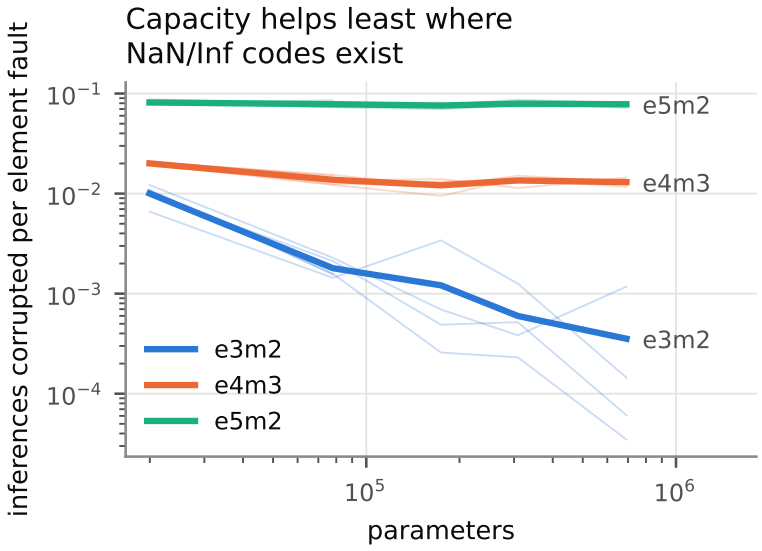


Figure 7: Element-fault rate against parameter count for ResNet8 at five widths. Faint lines are single networks, bold lines means.

Margins, not mantissa width. Among NaN-free formats, perturbation damage follows the network's sensitivity $s_i = |\partial m/\partial w_i| \cdot \mathrm{rms}/m$, where m is the top-two logit margin ($r = +0.93$ across formats), not the mantissa width ($r = -0.25$).

Free guards. The stage-2 checks flag only 3.5–4.1% of faults, but those faults carry 85% of the damage (Table 8). Guard G1 alone, tested on every network by injecting every element fault that lands on a special code, cuts element damage by at least 29× and up to 4100× on RepVGG and the Transformer, bringing `e5m2` down to the level of `e3m2`; on ResNet8 the cut is 3.9–53×, because a small network is also fragile to ordinary perturbations. The datapath matters as well: an E1 fault has damage 1.00 when the ReLU passes NaN and 0.085 when it zeroes it, while S2 stays at 0.97–0.98 because it produces $+\infty$, which passes any ReLU; only the scale check stops it.

Table 8: Read-time checks and guards on the exhaustive ResNet8 campaigns: share of damage each check flags, and the failure rate with and without the guards.

	e3m2	e4m3
D1 element code is NaN/Inf	–	52.2%
D2 scale outside its clean band	85.7%	32.1%
D1 + D2 + D3 together	85.8%	84.5%
Rate, no guard	0.00878	0.0236
Rate, G1 + G2	0.00206 (4.3× lower)	0.00454 (5.2× lower)

5.8. A harder task: CIFAR-100

All networks were retrained on CIFAR-100 and every headline result replicates: formats matched in accuracy, the order e3m2 < e4m3 < e5m2 on all nine networks, scale worse than element everywhere (1.7–3600×), and NaN codes carrying 94–100% of e4m3 and e5m2 element damage on the larger models. What changes is the floor. Smaller decision margins make the NaN-free formats 3–22× more vulnerable, while e4m3 and e5m2 barely move, so the gap between formats narrows (on the Transformer, e4m3/e3m2 falls from 461× to 19×). The guard still cuts damage 13–1387× on RepVGG and the Transformer.

5.9. New experiment E29: case-guided campaigns

E29 applies the flip-case algorithm to networks without an exhaustive campaign: it plans the campaign with the estimator of Section 3.4 (60 faults per stratum on ResNet8, 25 on RepVGG-A0) and scores each fault with no protection, with G1+G2, and with a NaN-zeroing ReLU.

Table 9: E29. "Faults" is the exact fault-space size, counted without injection. Variance reduction is against a uniform campaign of the same size. "Catastr. share" is the share of faults / share of damage in cases E1, S1, S2; "G1+G2" is how much the guards cut the rate.

Network	Fmt	Faults	Injected	Rate ± SE	Var. red.	Catastr. share	G1+G2
ResNet8, C-10	e3m2	483,904	1,020	0.00880 ± 0.00038	41×	0.5% / 57%	4.1×
ResNet8, C-10	e4m3	638,624	881	0.02347 ± 0.00043	108×	1.6% / 69%	5.0×
ResNet8, C-100	e3m2	519,904	1,020	0.0152 ± 0.0008	11×	0.5% / 33%	1.9×
ResNet8, C-100	e4m3	686,144	862	0.0251 ± 0.0005	75×	1.6% / 63%	4.1×
ResNet8, C-100	e5m2	686,144	724	0.0863 ± 0.0007	199×	7.6% / 88%	12×
RepVGG-A0, C-10	e3m2	43,980,128	425	0.00494 ± 0.00021	210×*	0.5% / 88%	258×*
RepVGG-A0, C-10	e4m3	58,053,504	450	0.01165 ± 0.00018	724×*	1.2% / 95%	1030×*

* No injected ordinary element fault (E2–E6) changed a prediction, so these figures are optimistic: the remaining rate is below what 25 faults per stratum can resolve.

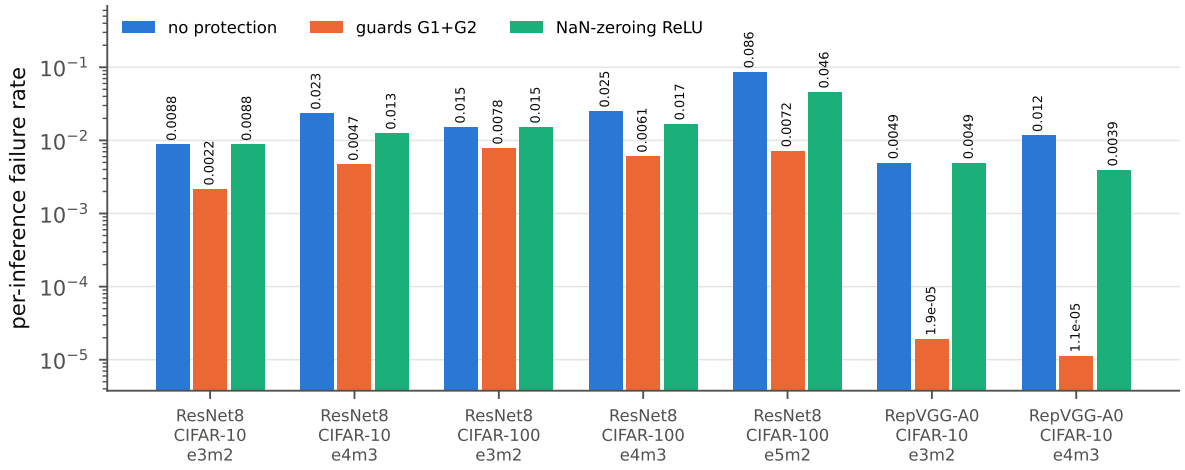


Figure 8: Per-inference failure rate in E29 (log scale): no protection, guards G1+G2, and a NaN-zeroing ReLU.

- **Cheap and accurate.** On ResNet8, about 1,000 injections give the exhaustive rate within 0.2 standard errors (0.00880 vs 0.00878; 0.02347 vs 0.02356), where the exhaustive campaigns needed 484–639 thousand. The variance is 11–724× smaller than uniform sampling of the same size.
- **The predictions carry over.** Every catastrophic fault was $0 \to 1$ in all 7 runs, and on RepVGG-A0 the catastrophic cases are 0.5–1.2% of faults but 88–95% of damage: a bigger network absorbs small errors, not NaN, ∞ or a 2^{128} scale.
- **The guards carry over.** G1+G2 cut the rate 4–12× on ResNet8 (only 1.9× on the low-accuracy CIFAR-100 `e3m2` model, where ordinary errors matter more) and by over 250× on RepVGG-A0. The ReLU helps only formats with NaN codes (1.5–3×).
- **Catastrophic can be silent.** On RepVGG-A0 `e3m2`, S2 flips damage 87% of predictions but only 8% give a non-finite output, so an output NaN check misses them. The scale check D2 catches all of them.

6. Methodological Findings

- **The usual SDC metric hides real differences.** Replaying one fixed list of 1,500 faults against nine sets of 200 images, the "any image changed" rate moved by up to 7× its own confidence width and tracked each set's smallest margin (Spearman +0.80 to +0.93). It called `e3m2` and `e4m3` equal and reversed their order between image sets. Scored per inference, the same faults gave 5.6–7.6×, stable across sets. Several early conclusions of this project were artefacts of this metric and were withdrawn.
- **Single-bit injection is enough.** Injecting every single- and double-bit pattern of 2,000 words per format (390,000 injections), a double flip does 1.04–1.55× the damage of a single flip and changes no conclusion.
- **Enumerate, do not learn.** Strata built from the exactly enumerated $|v' - v|$ of the MX code table cut variance 48–65×; TreeFI-style intervals learned from a pilot never beat uniform sampling. The case strata of E29 go further because they also separate the catastrophic cases.

7. Design Guidance

1. **Protect the shared scale first.** It is 2–11% of storage and a scale fault typically does 13 to over 1000× the damage of an element fault (only 1.7–4.3× for `e5m2`, whose element faults are already NaN-dominated). If only some scale bits can be protected, protect those normally at 0, whose flips grow the block.
2. **Neutralise special codes.** Prefer element formats without NaN/Inf, or use guards G1 and G2, which need only a comparator on the read path.
3. **Do not judge reliability by accuracy or bit width.** Equal-accuracy formats differ by orders of magnitude.
4. **Treat block size as a trade-off.** Large blocks are safe only with protected scales.
5. **Model the flip direction and protect early layers.** $0 \to 1$ flips carry almost all the risk, and the small stem layer is the most fragile.
6. **For campaigns:** report the per-inference rate, inject single-bit faults, and stratify by case.

8. Conclusion

This project built and applied a reliability analysis methodology for microscaling-based DNNs: a bit-accurate fault injector, validated against 1.1 million exhaustive injections, and a flip-case algorithm that explains how each fault propagates. Across formats, block sizes, tensor types and fault locations, on CNN and Transformer models, the central hypothesis holds on every network: reliability varies by orders of magnitude between MX formats of equal accuracy, and faults in the shared scale behave very differently from faults in elements, typically 13× to more than 1000× worse.

The cause is a small, exactly known set of flips. Flips into NaN/Inf codes and scale overflows are under 2% of stored bits (in `e3m2` and `e4m3`) but cause most failures, all of them are $0 \to 1$ flips, and model capacity cannot absorb them. Two read-time guards that cost no memory remove most of this damage, and a campaign planned by case measures a network with about a thousand injections instead of hundreds of thousands. In short: in an MX network, protect the scale and the few flips the algorithm names, and the rest matters far less.

9. Limitations and Future Work

- **ImageNet and pretrained Transformers.** DeiT on ImageNet was planned but ImageNet was not reachable. The Transformer here is a small ViT at DeiT-Tiny width trained on CIFAR, and it reaches only 47–49% on CIFAR-100. The narrowing gap on CIFAR-100 suggests absolute gaps on ImageNet-scale models may be smaller; the directions of the effects are the more transferable result. A case-guided campaign makes this affordable.
- **Fault model.** Only stored values are covered. Accumulators, partial sums, biases, normalisation and control logic are not, and faults that pile up between weight reloads were not measured.

- **Hardware.** The guards were simulated in software; they should be validated on an FPGA or accelerator model, together with other datapaths (saturating overflow, FP16 accumulation).
- **Model scale.** The largest network has 7 million parameters; large language models and MXINT8 were not studied.

Code and data. The `mxfi` library, all experiment scripts, trained networks and raw campaign outputs are at https://github.com/peeyushagarwal2004/CS496_UGP.

Acknowledgements

I thank my supervisor, Prof. Urbi Chatterjee, for the problem statement and guidance throughout this project. I also thank Vishesh Mishra for guiding me throughout the project, and IIT Kanpur for access to its GPU cluster.

References

[1] B. D. Rouhani et al., "Microscaling data formats for deep learning," *arXiv:2310.10537*, 2023.

[2] Open Compute Project, "OCP Microscaling Formats (MX) Specification, Version 1.0," 2023.

[3] G. Li, S. K. S. Hari, M. Sullivan, T. Tsai, K. Pattabiraman, J. Emer, and S. W. Keckler, "Understanding error propagation in deep learning neural network (DNN) accelerators and applications," in *Proc. SC*, 2017.

[4] B. Reagen et al., "Ares: A framework for quantifying the resilience of deep neural networks," in *Proc. DAC*, 2018.

[5] A. Mahmoud et al., "PyTorchFI: A runtime perturbation tool for DNNs," in *Proc. DSN Workshops*, 2020.

[6] R. Leveugle, A. Calvez, P. Maistri, and P. Vanhauwaert, "Statistical fault injection: Quantified error and confidence," in *Proc. DATE*, 2009.

[7] TreeFI: value-aware statistical fault injection for DNNs, in *Proc. ICCAD*, 2026.

[8] A. S. Rakin, Z. He, and D. Fan, "Bit-flip attack: Crushing neural network with progressive bit search," in *Proc. ICCV*, 2019.

[9] S. Hong, P. Frigo, Y. Kaya, C. Giuffrida, and T. Dumitraş, "Terminal brain damage: Exposing the graceless degradation in deep neural networks under hardware fault attacks," in *Proc. USENIX Security*, 2019.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, 2016.

[11] C. Banbury et al., "MLPerf Tiny benchmark," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.

[12] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, "RepVGG: Making VGG-style ConvNets great again," in *Proc. CVPR*, 2021.

[13] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021.

[14] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *Proc. ICML*, 2021.